

DSA Trees

Complete Pattern Notes
Self-Study & Last-Minute Revision Guide

Tree Patterns — Overview

Pattern	Core Idea	Key Problems
DFS Hard Variants	3 questions: what from children, compute at node, return to parent	Path Sum III, Distance K, Tree Cameras
BFS Hard Variants	Level-by-level with size snapshot. Track extra info per node in queue	Right View, Max Width, Vertical Order
Height/Diameter/Path	Bottom-up: compute at node using both children, return single value up	Max Path Sum, Diameter, Balanced Tree
Bottom-up DFS (TreeDP)	Return MULTIPLE values upward — as many as parent needs	House Robber III, Distribute Coins, Max BST
LCA	Both subtrees non-null = LCA found. BST: use value comparison	LCA 236/235, Deepest Leaves, Distance K
BST Hard Variants	In-order = sorted. Validate with min/max bounds. Prune with BST property	Validate, Kth Smallest, Delete, Range Sum
Tree Construction	Preorder root + inorder split. Postorder root from end	Build from Pre+In, Post+In, Sorted Array
Serialization	Add null markers to single traversal for unique reconstruction	Serialize/Deserialize 297

The one insight that unlocks hard tree problems: most hard problems = post-order DFS returning information bottom-up from children to parent.

Pattern 1 — DFS Hard Variants

Core Idea: At each node, answer 3 questions: (1) What do I need FROM children? (2) What do I compute AT this node? (3) What do I RETURN to parent?

The 3 Questions Framework

Problem	From Children	At Node	Return to Parent
Path Sum III	Prefix sums seen so far	Count paths ending here	Updated prefix map
Longest Same Path	Chain length from each child	leftLen + rightLen	max(leftLen, rightLen)
Tree Cameras	State (0/1/2) of children	Decide camera placement	State of current node
Distance K	Parent map (preprocessing)	BFS from target after DFS	N/A

Path Sum III — Prefix Sum + Hashmap (Leetcode 437)

Key insight: Same as array prefix sum. If $\text{currentSum} - \text{target}$ exists as a previous prefix sum, there's a valid path ending at current node.

```
int dfs(node, currSum, target, prefixCount){
    currSum += node->val;
    int count = prefixCount[currSum - target];
    prefixCount[currSum]++;
    count += dfs(left, ...) + dfs(right, ...);
    prefixCount[currSum]--; // CRITICAL: backtrack
    return count;
}
```

// Initialize: prefixCount[0] = 1 (empty path has sum 0)

Why decrement after recursion: hashmap tracks prefix sums on CURRENT root-to-node path only. When backtracking, remove current node's contribution so siblings don't see it.

Return max vs Return sum — The Critical Rule

```
// Computing answer AT node (path through both sides):
ans = max(ans, leftLen + rightLen);

// Returning TO parent (path can only go ONE direction up):
return max(leftLen, rightLen);
```

Path going to parent can only branch in ONE direction. The full leftLen+rightLen is the local answer. Only max goes up.

Tree Cameras — 3-State Bottom-up (Leetcode 968)

State	Meaning	When
0	Not covered — needs parent to place camera	Both children covered but no camera nearby

State	Meaning	When
1	Covered — no camera here	A child has a camera covering this node
2	Has camera	A child is uncovered — must place camera here

```

if(!node) return 1;           // null = covered
if(left==0 || right==0) { cameras++; return 2; } // child uncovered
if(left==2 || right==2) return 1;           // child has camera
return 0;                               // tell parent to cover us
// After dfs(root): if result==0, add one more camera at root

```

Convert Tree to Graph — When Needed

Signal: Problem requires traversal in BOTH directions (up and down). Trees only naturally support downward traversal.

```

// Step 1: Build parent map via BFS/DFS
unordered_map<TreeNode*, TreeNode*> parent;
parent[root] = nullptr;
// ... fill parent map for all nodes

// Step 2: BFS from target treating tree as undirected graph
// visit left child, right child, AND parent[node]
// use visited set to avoid going back

```

Solved Problems

Problem	LC#	Key Insight
Path Sum III	437	Prefix sum + hashmap. Decrement after recursion (backtrack).
Longest Path Same Value	2246	Pass direction lengths top-down, reset opposite direction to 0.
Binary Tree Cameras	968	3 states. Null=covered. Place camera when child=uncovered.
All Nodes Distance K	863	Parent map via BFS, then BFS from target as undirected graph.

Pattern 2 — BFS Hard Variants

Core Idea: BFS template with level-size snapshot. The key is what extra information to carry per node in the queue (index, row/col, state).

Universal BFS Template

```
queue<TreeNode*> q;
q.push(root);
while(!q.empty()){
    int size = q.size();          // snapshot current level size
    for(int i=0; i<size; i++){
        TreeNode* node = q.front(); q.pop();
        // i==0: leftmost node, i==size-1: rightmost node
        if(node->left) q.push(node->left);
        if(node->right) q.push(node->right);
    }
    // post-level processing here
}
```

The int size = q.size() snapshot is critical — captures exactly how many nodes are at the current level before you start adding the next level.

Maximum Width — Index as Heap Array (Leetcode 662)

Key insight: Index nodes like a heap: left child = $2*i$, right child = $2*i+1$. Width = last_index - first_index + 1.

```
queue<pair<TreeNode*, long long>> q;    // node + index
q.push({root, 0});
while(!q.empty()){
    int size=q.size();
    long long first=q.front().second;    // leftmost index
    for(int i=0; i<size; i++){
        auto [node, idx]=q.front(); q.pop();
        idx -= first;                    // normalize to prevent overflow
        if(node->left) q.push({node->left, 2*idx});
        if(node->right) q.push({node->right, 2*idx+1});
    }
    maxWidth = max(maxWidth, last - first + 1);
}
```

Normalize idx -= first at each level to prevent long long overflow. Indices grow exponentially — subtracting leftmost resets to 0 while preserving correct widths.

Vertical Order — Track Row and Column (Leetcode 987)

```
map<int, map<int, multiset<int>>>> nodes; // col -> row -> values
queue<tuple<TreeNode*,int,int>> q;         // node, col, row
```

```
q.push({root, 0, 0});  
// left child: col-1, right child: col+1, row+1 always  
// map keeps cols sorted, multiset handles duplicate values at same pos
```

Solved Problems

Problem	LC#	Key Insight
Right Side View	199	Last node at each level (i == size-1)
Maximum Width	662	Heap indexing + normalize each level to prevent overflow
Next Right Pointers	116	curr->next = q.front() (next in queue = next at same level)
Vertical Order Traversal	987	map+multiset for sorted col/row/value. Queue stores (node,col,row).

Pattern 3 — Height / Diameter / Path Sum

Core Idea: Bottom-up DFS. What you compute AT the node (global answer) and what you RETURN to parent are often DIFFERENT.

The Critical Distinction

Problem	Base Case	AT Node (global ans)	RETURN to Parent
Max Depth	0	N/A	$\max(L, R) + 1$
Diameter	0	$L + R$	$\max(L, R) + 1$
Max Path Sum	0	$\max(0, L) + \max(0, R) + \text{val}$	$\max(0, L, R) + \text{val}$
Balanced Tree	0	check $\text{abs}(L - R) > 1$	$\max(L, R) + 1$ or -1
Path Sum II	N/A	record if $\text{leaf} + \text{sum} == \text{target}$	backtrack <code>pop_back</code>

Max Path Sum — Handle Negatives (Leetcode 124)

```
int left = max(0, dfs(node->left)); // ignore if negative
int right = max(0, dfs(node->right)); // ignore if negative
ans = max(ans, left + right + node->val); // path through node
return max(left, right) + node->val; // best arm to parent
// Initialize ans = INT_MIN (all nodes could be negative)
```

Balanced Tree — Sentinel -1 (Leetcode 110)

```
int dfs(TreeNode* node){
    if(!node) return 0;
    int left=dfs(node->left), right=dfs(node->right);
    if(left==-1 || right==-1) return -1; // propagate unbalanced
    if(abs(left-right) > 1) return -1; // unbalanced here
    return max(left, right)+1;
}
```

Top-down vs Bottom-up

Use	When	Key Feature
Top-down (pass info down)	Collect paths root-to-leaf, path sum with target	backtrack with <code>pop_back</code> after recursion
Bottom-up (return info up)	Compute using BOTH subtrees, diameter, path sum	global variable for answer, different return value

Solved Problems

Problem	LC#	Key Insight
Binary Tree Max Path Sum	124	$\max(0, L) + \max(0, R) + \text{val}$ at node. Return $\max(0, L, R) + \text{val}$. $\text{ans} = \text{INT_MIN}$.
Path Sum II	113	Top-down. pass remaining sum. pop_back to backtrack.
Balanced Binary Tree	110	Return -1 as sentinel. Propagates up automatically.
Longest ZigZag Path	1372	Pass (leftLen, rightLen) top-down. Reset opposite direction to 0.

Pattern 4 — Bottom-up DFS (Tree DP)

*Core Idea: Return MULTIPLE values upward — as many as the parent needs to make its optimal decision.
Simple bottom-up returns one value; Tree DP returns several.*

Simple Bottom-up vs Tree DP

	Simple Bottom-up	Tree DP
Return value	Single value (height, sum)	Multiple values (states, validations)
Use when	One thing needed from child	Parent needs multiple things from child
Return type	int	pair, tuple, struct, or vector

House Robber III — Two States (Leetcode 337)

```
pair<int,int> dfs(TreeNode* node){    // {rob, notRob}
    if(!node) return {0, 0};
    auto [lR, lNR] = dfs(node->left);
    auto [rR, rNR] = dfs(node->right);
    int rob    = node->val + lNR + rNR;        // rob this, skip children
    int notRob = max(lR,lNR) + max(rR,rNR);    // skip this, best from each child
    return {rob, notRob};
}
// answer = max(rob, notRob) at root
```

Distribute Coins — Excess Flow (Leetcode 979)

```
int dfs(TreeNode* node){
    if(!node) return 0;
    int left=dfs(node->left), right=dfs(node->right);
    moves += abs(left) + abs(right);    // coins flowing through edges
    return node->val + left + right - 1;    // excess after keeping 1
}
```

Maximum Sum BST — Four Values (Leetcode 1373)

```
// Return: {isBST, minVal, maxVal, sum}
if(!node) return {true, INT_MAX, INT_MIN, 0};
// Current node is valid BST if:
// both children BST AND left_max < node->val < right_min
if(lBST && rBST && lMax < node->val && node->val < rMin){
    int sum = lSum + rSum + node->val;
    maxSum = max(maxSum, sum);
    return {true, min(lMin,node->val), max(rMax,node->val), sum};
}
```



```
return {false, 0, 0, 0}; // not BST
```

Solved Problems

Problem	LC#	States Returned
House Robber III	337	(rob, notRob) — two choices affecting parent's decision
Binary Tree Coloring Game	1145	Three regions: left, right, parent. Win if any > n/2.
Distribute Coins	979	Excess coins. moves += abs(left) + abs(right).
Maximum Sum BST	1373	(isBST, minVal, maxVal, sum) — all needed for parent validation.

Pattern 5 — LCA (Lowest Common Ancestor)

Core Idea: If p is in left subtree and q in right subtree (or node == p or q), current node IS the LCA. If both in same subtree, recurse deeper.

Base LCA Algorithm (Leetcode 236)

```
TreeNode* lca(TreeNode* root, TreeNode* p, TreeNode* q){
    if(!root) return nullptr;
    if(root==p || root==q) return root;    // found one
    TreeNode* left = lca(root->left, p, q);
    TreeNode* right = lca(root->right, p, q);
    if(left && right) return root;          // p and q split here = LCA
    return left ? left : right;            // both in same subtree
}
```

LCA Variants

Variant	Key Insight	LC#
BST LCA	Both < root: go left. Both > root: go right. Split point = LCA.	235
With parent ptrs	Two pointer trick like linked list intersection.	1650
Deepest leaves LCA	Return (node, depth). Equal depth = LCA here. Deeper side = LCA deeper.	1123
Multiple nodes	Same algorithm, check against hashset instead of p and q.	General
Path between nodes	Build parent map + DFS/BFS treating tree as undirected graph.	2096

Distance Between Two Nodes

$$\text{dist}(p, q) = \text{depth}(p) + \text{depth}(q) - 2 * \text{depth}(\text{LCA}(p, q))$$

Solved Problems

Problem	LC#	Key Insight
LCA of Binary Tree	236	Both subtrees non-null = LCA. Else recurse into non-null side.
LCA of BST	235	BST property: go left/right based on values. Split = LCA.
LCA of Deepest Leaves	1123	Return (node, depth). Equal depth at node = LCA of deepest leaves.
Step-By-Step Directions	2096	Parent map + BFS from start as undirected graph to dest.

Pattern 6 — BST Hard Variants

Core Idea: BST property (left < node < right) enables $O(h)$ search, in-order = sorted sequence, and subtree pruning. Use these to avoid $O(n)$ full traversal.

BST Properties Cheat Sheet

Property	Use Case
Left < node < right	Validation, search, delete — prune half the tree at each step
In-order = sorted ascending	Kth smallest, median, sorted output
Reverse in-order = descending	Kth largest, greater sum tree
Middle of sorted = root	Build balanced BST from sorted array
Prune with value bounds	Range queries, validation with min/max

Validate BST — Min/Max Bounds (Leetcode 98)

Why checking only parent-child is wrong: A node in right subtree must be greater than ALL ancestors, not just its immediate parent.

```
bool validate(TreeNode* node, long minVal, long maxVal){
    if(!node) return true;
    if(node->val <= minVal || node->val >= maxVal) return false;
    return validate(node->left, minVal, node->val) &&
           validate(node->right, node->val, maxVal);
}
// Call: validate(root, LONG_MIN, LONG_MAX) -- use long not int
```

Delete Node — 3 Cases (Leetcode 450)

Case	Condition	Action
1	No children	Just remove node, return nullptr
2	One child	Replace node with that child
3	Two children	Find inorder successor (leftmost in right subtree), copy its value, delete successor

Solved Problems

Problem	LC#	Key Insight
Validate BST	98	Pass min/max bounds as long. Check both bounds at every node.
Kth Smallest	230	In-order traversal. Decrement k at each node. Stop when k==0.

Problem	LC#	Key Insight
Delete Node in BST	450	3 cases. Inorder successor = leftmost node in right subtree.
BST to Greater Sum Tree	1038	Reverse in-order (right->node->left) with running sum accumulator.
Range Sum of BST	938	Prune: skip left if node<=low, skip right if node>=high.

Pattern 7 — Tree Construction

Core Idea: Preorder gives root first. Inorder tells you what is left/right of that root. Together they uniquely define the tree.

Traversal Properties

Traversal	Order	Key Property
Preorder	root, left, right	FIRST element is always root
Inorder	left, root, right	Root SPLITS array into left and right subtrees
Postorder	left, right, root	LAST element is always root

Pair	Unique Tree?
Preorder + Inorder	YES - unique
Postorder + Inorder	YES - unique
Preorder + Postorder	NO - multiple trees possible

Build from Preorder + Inorder (Leetcode 105)

Key formula: $\text{leftSize} = \text{inRoot} - \text{inStart}$. Use running index into preorder (passed by reference) instead of tracking preorder ranges.

```
TreeNode* build(vector<int>& pre, unordered_map<int,int>& inMap,
               int& idx, int inStart, int inEnd){
    if(inStart > inEnd) return nullptr;
    int rootVal = pre[idx++];
    TreeNode* root = new TreeNode(rootVal);
    int inRoot = inMap[rootVal];
    root->left = build(pre, inMap, idx, inStart, inRoot-1);
    root->right = build(pre, inMap, idx, inRoot+1, inEnd);
    return root;
}
```

// Hashmap for inorder index: $O(1)$ lookup vs $O(n)$ scanning

Build from Postorder + Inorder (Leetcode 106)

```
// Same as 105 but:
// 1. idx starts at END of postorder, decrements (idx--)
// 2. Build RIGHT subtree BEFORE left (postorder is left,right,ROOT
//    so reverse = ROOT,right,left)
root->right = build(post, inMap, idx, inRoot+1, inEnd);
root->left = build(post, inMap, idx, inStart, inRoot-1);
```

Sorted Array to Balanced BST (Leetcode 108)

```
TreeNode* build(vector<int>& nums, int lo, int hi){  
    if(lo > hi) return nullptr;  
    int mid = lo + (hi-lo)/2;  
    TreeNode* node = new TreeNode(nums[mid]);  
    node->left = build(nums, lo, mid-1);  
    node->right = build(nums, mid+1, hi);  
    return node;  
}
```

Solved Problems

Problem	LC#	Key Insight
Build from Preorder+Inorder	105	Running idx++ into preorder. Hashmap for O(1) inorder lookup.
Build from Postorder+Inorder	106	Running idx-- from end. Build RIGHT before LEFT.
Sorted Array to BST	108	Middle element = root. Guarantees height balance.
BST from Preorder	1008	Find split point where elements exceed root value.

Pattern 8 — Serialization

Core Idea: Add null markers to a single traversal sequence. This preserves enough structure to uniquely reconstruct the tree without needing a second traversal.

Why Null Markers Work

```
Tree:      1
           / \
          2   3
           / \
          4   5

Preorder with nulls: '1,2,null,null,3,4,null,null,5,null,null'
This UNIQUELY reconstructs the tree -- no second traversal needed
```

DFS Serialization (Preorder)

```
// Serialize
string serialize(TreeNode* root){
    if(!root) return 'null,';
    return to_string(root->val)+','+serialize(root->left)+serialize(root->right);
}

// Deserialize -- queue gives elements in preorder
TreeNode* build(queue<string>& q){
    string val=q.front(); q.pop();
    if(val=='null') return nullptr;
    TreeNode* node=new TreeNode(stoi(val));
    node->left  = build(q);
    node->right = build(q);
    return node;
}
```

DFS vs BFS Serialization

	DFS (Preorder)	BFS (Level Order)
Code complexity	Simpler — natural recursion	More complex — two queues
Reconstruction	Recursive build with queue	Level-by-level queue processing
Use when	General purpose	Level-order format required

Solved Problems

Problem	LC#	Key Insight
Serialize and Deserialize Binary Tree	297	Add null markers. Preorder DFS serialize. Queue for reconstruct.

Quick Reference — All 8 Tree Patterns

Pattern	Time	Key Signal Words
DFS Hard Variants	$O(n)$	path sum, cameras, distance K, same value path
BFS Hard Variants	$O(n)$	right view, width, vertical order, level average
Height/Diameter/Path	$O(n)$	diameter, max path sum, balanced, path sum root-to-leaf
Tree DP	$O(n)$	rob nodes, distribute, optimal assignment on tree
LCA	$O(n)/O(h)$	lowest common ancestor, distance between nodes
BST Hard Variants	$O(h)$	validate BST, kth smallest, delete, range sum
Tree Construction	$O(n)$	construct from traversals, sorted array to BST
Serialization	$O(n)$	serialize, deserialize, encode, decode tree

Most Important Rules

- 3 Questions: What from children? What at node? What return to parent? Answer these before coding any DFS problem.
- Global ans vs return value: compute path-through-node locally (L+R), return single arm to parent ($\max(L,R)$).
- Negative subtrees: use $\max(0, \text{dfs}(\text{child}))$ to ignore subtrees that hurt the answer (max path sum).
- Sentinel -1: use -1 as return value to signal invalid state propagating upward (balanced tree check).
- Tree DP: return as many values as parent needs. pair for 2, struct for 4+.
- LCA: both subtrees non-null = LCA here. BST: use value comparison to prune direction.
- BST validation: always pass min/max bounds as long (not int) to handle `INT_MIN/INT_MAX` node values.
- Construction: preorder `idx++` runs left to right. Postorder `idx--` runs right to left (build right before left).
- Null markers in serialization: single traversal with nulls uniquely reconstructs tree. No second traversal needed.

— End of Notes —